Nama : Gregorius Septiano Ariadi

NIM : 24060124120026

Kelas : Struktur Data / E

Cara pembentukan proses sesuai NIM:

NIM: 24060124**120026**

Lima digit terakhir: **20026**

Aturan:

- Setiap digit mewakili Burst Time untuk satu proses (P1..P5)
- Jika ada digit 0, maka diganti dengan digit sebelumnya + 3

Perhitungan:

P1: 2 → langsung = 2

P2: 0 → 2 + 3 = 5

P3: 0 → 0 sebelumnya (5) + 3 = 8

P4: 2 → langsung = 2

P5: 6 → langsung = 6

```
DEFINISI & SPESIFIKASI TIPE & PROTOTIPE

type Proses : < IdProses: character, BurstTime: integer >

type tqueue : < wadah: array [1..5] of Proses, head: integer, tail: integer >

procedure makeProses (output P: Proses, input x: character, y: integer)
{ I.S.: - }
{ F.S.: P terdefinisi, kosong }
{ Proses: mengisi nilai IdProses dengan x dan BurstTime dengan y }

procedure SetIDProses (input/output P: Proses, input id: character)
{ I.S.: P terdefinisi, id terdefinisi }
{ F.S.: nilai id pada Proses P berubah sesuai parameter id }
{ Proses: mengganti IDProses dari Proses }

procedure SetBurstTime (input/output P: Proses, input y: integer)
{ I.S.: P terdefinisi, y terdefinisi }
{ F.S.: nilai BurstTime pada Proses P berubah sesuai parameter y }
{ Proses: mengganti BurstTime dari Proses }

function GetIDProses(P: Proses) → character
{mengembalikan nilai komponen IdProses P}
```

```
function GetBurstTime(P: Proses) → integer
{mengembalikan nilai komponen BurstTime P}

function Head(Q:tqueue) → integer
{mengembalikan elemen terdepan antrian Q}

function Tail(Q:tqueue) → integer
{mengembalikan elemen terakhir antrian Q}

procedure CreateQueue (output Q:tQueue)
{ I.S.: - }
{ F.S.: Q terdefinisi, kosong }
{ Proses: mengisi IdProses = '-' dan BurstTime = -999, head=tail=0 }

function IsEmptyQueue(Q:tqueue) → boolean
{Mengembalikan true jika Queue kosong}

function IsFullQueue(Q:tqueue) → boolean
{Mengembalikan true jika Queue penuh}

procedure Enqueue (input/output Q:tqueue, input P: Proses)
{ I.S.: Q dan P terdefinisi }
{ F.S.: elemen P ditambahkan ke antrian jika belum penuh }

void Dequeue (input/output Q:tqueue)
{ I.S.: Q terdefinisi, mungkin kosong }
{ F.S.: jika tidak kosong, mengeluarkan elemen pertama (head) dan menggeser
elemen lainnya }

procedure PrintQueue (input/output Q:tqueue)
{ I.S.: Q terdefinisi }
{ F.S.: menampilkan seluruh elemen antrian ke layar }

procedure SortByBurstTime (input/output Q:tqueue)
{ I.S.: Q berisi elemen }
{ F.S.: antrian diurutkan berdasarkan Burst Time dari kecil ke besar }
```

SORT BY BURST TIME

```
procedure SortByBurstTime (input/output Q:tqueue)
{ I.S.: Q berisi elemen }
{ F.S.: antrian diurutkan berdasarkan Burst Time dari kecil ke besar }
{ Sebagai procedure utama untuk mengurutkan tqueue berdasarkan atribut
BurstTime yang nantinya akan tinggal di dequeue oleh procedure dequeue }
```

REALISASI

Kamus Lokal

```
 use Proses
 i, j : integer
 temp : Proses
```

Algoritma

```
      i traversal [head(Q)..tail(Q)-1]
        j traversal [i+1..tail(Q)]
```

```
            if (GetBurstTime(Q.wadah[i]) > GetBurstTime(Q.wadah[j])) then
                temp ← Q.wadah[i]
                Q.wadah[i] ← Q.wadah[j]
                Q.wadah[j] ← temp
```

APLIKASI

Program MainProses
{*program untuk mensimulasikan penjadwalan proses pada CPU menggunakan algoritma penjadwalan SJF*}

Kamus

```
      use Proses
      P1, P2, P3, P4, P5, keluar    : Proses
      Q                             : tqueue
      waktuMulai, waktuSelesai, i   : integer
```

Algoritma

```
      {Membuat proses dari P1 sampai P5 dengan NIM terakhir 20026}
      MakeProses(P1, 'A', 2)
      MakeProses(P2, 'B', 5)
      MakeProses(P3, 'C', 8)
      MakeProses(P4, 'D', 2)
      MakeProses(P5, 'E', 6)
      CreateQueue(Q)

      {Memasukkan proses dari P1 sampai P5 ke dalam queue}
      Enqueue(Q, P1)
      Enqueue(Q, P2)
      Enqueue(Q, P3)
      Enqueue(Q, P4)
      Enqueue(Q, P5)
      PrintQueue(Q)

      {Sorting Queue berdasarkan BurstTime dari Proses)
      SortByBurstTime(Q)
      PrintQueue(Q)

      {Memulai simulasi pengambilan proses}
      waktuMulai ← 0
      i traversal [head(Q)..tail(Q)]
          waktuSelesai ← waktuMulai + GetBurstTime(Q.wadah[i])
          output(waktuMulai, waktuSelesai)
          waktuMulai ← waktuSelesai
      {Memulai Dequeue Proses}
      while (not isEmptyQueue(Q)) do
          keluar ← Dequeue(Q)
          PrintQueue(Q)
```

Screenshoot Code

```
/* Procedure SortByBurstTime(Q:tqueue)
   I.S.: Q berisi elemen
   F.S.: antrian diurutkan berdasarkan Burst Time dari kecil ke besar */
void SortByBurstTime(tqueue *Q) {
    /*Kamus Lokal*/
    int i, j;
    for (i = (*Q).head; i < (*Q).tail; i++) {
        for (j = i + 1; j <= (*Q).tail; j++) {
            if (GetBurstTime((*Q).wadah[i]) > GetBurstTime((*Q).wadah[j])) {
                Proses temp = (*Q).wadah[i];
                (*Q).wadah[i] = (*Q).wadah[j];
                (*Q).wadah[j] = temp;
            }
        }
    }
}
```

```c
/* Program   : main.c */
/* Deskripsi : Program utama (driver) digunakan untuk mensimulasikan penjadwalan proses CPU denga
n              memanfaatkan ADT Proses dan ADT Queue. */
/* NIM/Nama  : Gregorius Septiano Ariadi / 24060124120026 */
/* Tanggal   : Jumat, 10 Oktober 2025 */
/**********************************************/

#include <stdio.h>
#include "proses.c"

int main() {
    /* Kamus */
    tqueue Q;
    Proses P1, P2, P3, P4, P5, keluar;
    int waktuMulai, waktuSelesai, i;

    /* Algoritma */
    printf("Simulasi Penjadwalan Proses CPU (SJF)\n\n");

    printf("=== Uji 1: Inisialisasi Queue ===\n");
    CreateQueue(&Q);
    PrintQueue(Q);
    printf("Queue kosong? %d\n", IsEmptyQueue(Q));
    printf("Queue penuh?  %d\n\n", IsFullQueue(Q));

    printf("=== Uji 2: Pembuatan Proses ===\n");
    MakeProses(&P1, 'A', 2);
    MakeProses(&P2, 'B', 5);
    MakeProses(&P3, 'C', 8);
    MakeProses(&P4, 'D', 2);
    MakeProses(&P5, 'E', 6);
    printf("Proses dibuat: A(2), B(5), C(8), D(2), E(6)\n\n");

    printf("=== Uji 3: Enqueue Proses ===\n");
    Enqueue(&Q, P1);
    Enqueue(&Q, P2);
    Enqueue(&Q, P3);
    Enqueue(&Q, P4);
    Enqueue(&Q, P5);
    PrintQueue(Q);
    printf("Head = %d, Tail = %d\n", head(Q), tail(Q));
    printf("Queue penuh? %d\n\n", IsFullQueue(Q));

    printf("=== Uji 4: Urutkan Berdasarkan Burst Time (SJF) ===\n");
    SortByBurstTime(&Q);
    PrintQueue(Q);
    printf("\n");

    printf("=== Uji 5: Simulasi Eksekusi Proses ===\n");
    waktuMulai = 0;
    for (i = head(Q); i <= tail(Q); i++) {
        waktuSelesai = waktuMulai + GetBurstTime(Q.wadah[i]);
        printf("Proses %c dimulai %d, selesai %d (Burst: %d)\n",
               GetIDProses(Q.wadah[i]),
               waktuMulai, waktuSelesai, GetBurstTime(Q.wadah[i]));
        waktuMulai = waktuSelesai;
   }

    printf("\n=== Uji 6: Dequeue Semua Proses ===\n");
    while (!IsEmptyQueue(Q)) {
        keluar = Dequeue(&Q);
        printf("Keluar: %c | Sisa antrian: ", GetIDProses(keluar));
        PrintQueue(Q);
   }
    printf("Queue kosong sekarang.\n\n");

    printf("=== Uji 7: Kondisi Akhir ===\n");
    printf("Queue kosong? %d\n", IsEmptyQueue(Q));
    printf("Queue penuh?  %d\n", IsFullQueue(Q));
    PrintQueue(Q);

    return 0;
}
```

```
Simulasi Penjadwalan Proses CPU (SJF)

=== Uji 1: Inisialisasi Queue ===
Queue kosong
Queue kosong? 1
Queue penuh?  0

=== Uji 2: Pembuatan Proses ===
Proses dibuat: A(2), B(5), C(8), D(2), E(6)

=== Uji 3: Enqueue Proses ===
Antrian proses:
Proses A | Burst Time: 2
Proses B | Burst Time: 5
Proses C | Burst Time: 8
Proses D | Burst Time: 2
Proses E | Burst Time: 6
Head = 1, Tail = 5
Queue penuh? 1

=== Uji 4: Urutkan Berdasarkan Burst Time (SJF) ===
Antrian proses:
Proses A | Burst Time: 2
Proses D | Burst Time: 2
Proses B | Burst Time: 5
Proses E | Burst Time: 6
Proses C | Burst Time: 8

=== Uji 5: Simulasi Eksekusi Proses ===
Proses A dimulai 0, selesai 2 (Burst: 2)
Proses D dimulai 2, selesai 4 (Burst: 2)
Proses B dimulai 4, selesai 9 (Burst: 5)
Proses E dimulai 9, selesai 15 (Burst: 6)
Proses C dimulai 15, selesai 23 (Burst: 8)
```

```
=== Uji 6: Dequeue Semua Proses ===
Keluar: A | Sisa antrian: Antrian proses:
Proses D | Burst Time: 2
Proses B | Burst Time: 5
Proses E | Burst Time: 6
Proses C | Burst Time: 8
Keluar: D | Sisa antrian: Antrian proses:
Proses B | Burst Time: 5
Proses E | Burst Time: 6
Proses C | Burst Time: 8
Keluar: B | Sisa antrian: Antrian proses:
Proses E | Burst Time: 6
Proses C | Burst Time: 8
Keluar: E | Sisa antrian: Antrian proses:
Proses C | Burst Time: 8
Keluar: C | Sisa antrian: Queue kosong
Queue kosong sekarang.

=== Uji 7: Kondisi Akhir ===
Queue kosong? 1
Queue penuh?  0
Queue kosong
```